

SapFlow Sensor Official

The following document outline the operation and recommendations for the SapFlow Sensor.

This project is heavily inspired and adapted from Dotmote Labs: [Sapflow Using the External Heat Ratio Method](#)

The following drive has the all proprietary files of the SapFlow sensor, as well as a back-up of the Dotmote Labs github repository in case of absolution: [SU_SapFlow](#) – only accessible with SU credentials.

Contents

SapFlow Sensor Official	1
Documentation	2
Operation	2
Analyzing the Datalog	3
Assembly	3
Recommendations	3
Real-Time Clock (RTC)	3
Power Optimization	4
Wireless Communications & Server	4
Battery Charging Circuit	5
Cost	5
Casing	5
Final Remarks	5
Datasheets	5
Revisions	5

Documentation

Operation

It is important that each device has an SD Card with the CONFIG.txt and README.txt files on.

The device has two main *states*:

1. Sensor Node
2. Gateway Node

To *change between states*, the reset button is pressed on the device.

LED indicators of states:

- State 1 -> LED will constantly be HIGH, to indicate server is running.
- State 2 -> LED will be HIGH for 10 sec. then pulse when measurements is done.

State 1: Sensor Node

- The device will boot, immediately reading CONFIG.txt, initializing parameters, and start to search for Wi-Fi SSID of Gateway.
- If the SSID is found, the datalog file is uploaded to server and devices continues to do measurements.
- If after 10 sec. it is not found, the device continues operation of measurements.
- After the measurement sequence has concluded, as stated by the CONFIG.txt parameters, the device goes to sleep.
- After sleep timer expires, this process repeats.

State 2: Gateway Node

- The device will boot, immediately reading CONFIG.txt, initializing parameters, and start a Wi-Fi access point with Gateway_ID SSID
- The device goes into a constant loop of client handling.
- To access the server, connect to the TJT Gateway SSID with the password (specified at below) and go to <http://tjt.local>
- Wait for approximately 15min to receive all the datalogs and refresh the page.
- To end this state the reset button must be pressed again.

Gateway SSID Password = “22786171”, this can only be modified in the source code for security purposes.

State Diagram

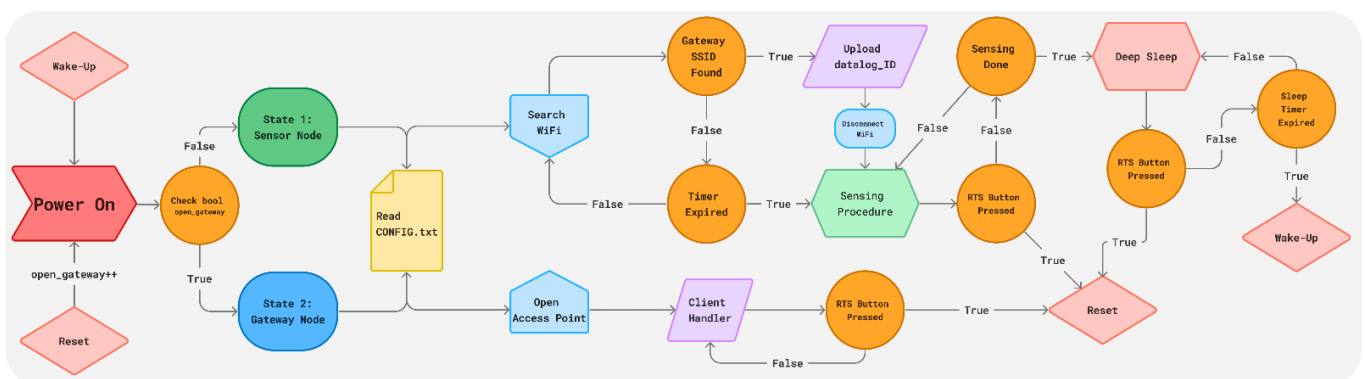


Figure 1: Device Operational State Diagram

Analyzing the Datalog

The datalog is outputted to a .csv file in comma delimited format as follow:

Item	Info
currentMillis	Milliseconds since temp. measuring started
batt_voltage	Battery voltage measure on power up
batteryPercentage	Voltage converted to precentage
temp1	Temperature measurement of sensor 1
temp2	Temperature measurement of sensor 2
heatPulse_state	State of sensing procedure: 0 = before heat pulse 1 = heat applied 2 = after heatpulse

Assembly

The device is designed modular for easier future changes, as this is still a prototype. It is important to that the D32 Pro's is mounted with the USB directed to the SapFlow Gauge male pins. Make sure that the RJ45 pins are connected correctly according the I2C pinout between the D32 Pro shield and the SapFlow Gauge. Also, make sure the Ethernet cable "clicks" when inserted

Recommendations

In this section I will share some considerations for future improvements.

Real-Time Clock (RTC)

One major flaw is the absence to record real-time timestamps. Although the microcontroller shield accommodates for this, there is **not** alluded to a RTC by the Dotmote Labs components list, and only when looking at the code or assembly. Three possible solutions:

1. An external RTC

As mentioned, the shield accommodates for this, however the RTC Dotmote recommends, the DS3231 Adafruit breakout, cost R335 per RTC through DigiKey. The reason DigiKey is the main reference point is due them being the only supplier of this RTC breakout as of writing, but also due to the pinout on the shield. The pinout used, is specific to this breakout (Vin, GND, SCL, SDA), and not many, if any other follow this pin order. The options are as follow:

- Buy the intended expensive RTC breakout.
- **Source a cheaper one, with the same pinout.**
- Buy the RTC module and design the breakout to fit.
- Redesign the microcontroller shield, to accommodate a more common pinout, either by SPI or I2C RTCs

Having an external RTC, will have minimal impact on software change (most is already written, just commented out.)

2. RTC Server

An easy alternative without additional components would be to connect configure the devices to connect through the internet to a RTC server, retrieving the current date and time. This however means expected location for use of the sensors need to have Wi-Fi connected

to the internet, which is not always the case. This will also implicate a slight change in software.

3. Manually Setting MCU Time

This method is not recommended, but if somehow an algorithm is thought-up to achieve even slightly near accurate date and time with every boot, go for it 🙄

Power Optimization

Power optimization was not the primary focus for the first version of the SapFlow sensor. Power consumption was also not measured, or tested, hence no estimation can be given at this stage of battery life. To accommodate possible poor static & dynamic power usage of the MCU, without hours invested into power optimization, a large LiPo of 5000mAh was opted for and the device does go into deep sleep. The following should be considered on revision:

1. Software

Power optimization in the software of the ESP32 should be thoroughly investigated, beyond deep sleep, to lessen power use.

2. Microcontroller Development Board

All MCU development boards have unique characteristics when it comes to static (when idling, leak current) and dynamic (operational) power usage. This also impacts the battery life of a device significantly. Hence, reconsider, do research and compare whether the D32 Pro is indeed well suited for low energy applications. The MCU itself should also be a factor in this choice, since the ESP32 only has one low-power coprocessor, whereas the newer ESP32-S3 has two.

Wireless Communications & Server

Dotmote Labs original code makes use MQTT with a Raspberry-Pi being the host and the D32 Pro being the clients. With much trouble their code could not be set up to work as intended. After deliberation on how to proceed it was decided to not complicate implementation by using a Raspberry-Pi, as this would mean an additional board per cluster, and a different power source than the easily connected battery the D32 Pro uses. Hence the custom HTTP server Gateway-Node implementation was constructed. Revision should consider the following:

1. Communication Technology Used

Investigation could be done to find a better method of utilizing wireless communication i.e. MQTT, Wi-Fi, 3G/4G, LoRa (which would mean a major design revisit). Also, perhaps revisit an R-Pi implementation if thought reasonable.

2. Server Implementation

The HTTP server is a more accessible way to gather and download all the datalog files of the devices than individually fetching and copying the files from the SD cards. Two additional functions to explore on the server would be:

- A button and method to zip and download all the datalogs in one press.
- If be that the CONFIG parameters needs change, a method to upload only one CONFIG.txt file and then do a redistribution to all the other nodes (i.e. check if file has changed and share with each node when joining the gateway).

A further option for easy data retrieval would be an API server, but would require again access to the internet, or major design revision of LoRa is chosen.

3. Range Testing & Error Handling

Range testing of the Wi-Fi capabilities has not been. It would be advised to further implement error handle, i.e. exception handling when HTTP error codes are received, when connection is dropped while a file is uploaded due to poor signal strength etc.

Battery Charging Circuit

An external battery charging bank could be a nice solution to bulk charge the batteries and swap them out with each other. LiPo battery chargers are quite cheap.

Cost

The biggest expense was in this project was using the D32 Pro microcontroller boards, as they are only sold by AliExpress and shipped from China. Even though the board itself was only ~R189, the shipping and custom duties accounted for half the price of acquiring them. Rather source other boards locally. This will however mean redesigning the shield and extra development time.

Casing

An environmentally robust case must still be designed for the microcontroller board.

Final Remarks

The primary outcome from this device, calculating sap flow, is heavily dependent on the accuracy of the temperature sensor measurement. Please know that there could be margin of error, specifically two sensors from one gauge in the same room temp, not 100% measuring the same temp.

Finally, the battery measurement, is a relative measurement based on the accuracy of ADC. Values may appear to fluctuate. This should not be of major concern. The battery level is mere an indication when to charge (if below < 20%).

Get stickers to label the boards and their corresponding SD cards. The CONFIG.txt must end with a new line delimiter (“\n”, i.e. an empty line).

Datasheets

To see the schematics of the sensor gauge or die microcontroller shield, please refer to the Dotmote Lab links mentioned on the cover page.

Battery

[Ful-Energy-Batteries/105570_5000mAh_Data_sheet_V3.pdf_at_master · microrobotics/Ful-Energy-Batteries \(github.com\)](#)

Lolin Wemos D32 Pro Microcontroller

[D32 Pro — WEMOS documentation](#)

Revisions

Version	Author	Date
1.0	Waldo Jordaan	03/05/2024